Why regenerate SignWriting OneD?

A SuttonSignWritingOneD.ttf already exists — Slevinski's official build at sutton-signwriting/font-ttf. Two things make it worth rebuilding:

1. Cubic → quadratic conversion loss

The source SVG glyphs in sutton-signwriting/font-db use cubic Bezier curves. TrueType only supports quadratic Beziers, so FontForge approximates each cubic with a sequence of quadratics during the .ttf export. The approximation has a tolerance of "about one emunit" (splineorder2.c) — visible in our earlier round of analysis as a ~1-pixel stroke wobble for any glyph compared between the cubic source and the TTF render.

2. Duplicated outlines for rotations & reflections

Every SignWriting symbol has 16 orientation variants (the last hex digit of the symbol key). In the upstream TTF every variant carries its own full outline. For the cardinal orientations (rot 0, 2, 4, 6, 8, A, C, E) the outline really is just a rotation/reflection of rot 0, and likewise the diagonals (1, 3, 5, 7, 9, B, D, F) are transforms of rot 1 — so 14 of every 16 glyphs can become TrueType composite glyphs that reference one base outline plus a 2×2 transform. Cuts the file by ~67 %.

Following pages walk through what the regeneration produces: the file size and codepoint coverage, the rotation encoding, side-by-side renders of the optimised symbols, and a list of glyphs that still don't quite match upstream so a human can decide whether to accept them.

SignWriting OneD — regeneration report

Comparison of the upstream Sutton SignWriting OneD font vs the font regenerated from font-db's cubic-Bezier SVG sources, with and without our ellipse-replacement optimization.

File size

variant	size		Δ
original (upstream	8,206,192 bytes	8013.9 KB	
new (no optimisatio	7,537,524 bytes	7360.9 KB	-8.1% vs original
new (ellipse + rota	3,130,476 bytes	3057.1 KB	-61.9% vs original

Glyph count

variant	glyphs
original (upstream OneD)	38,316 mapped codepoints
new (no optimisations)	38,316 mapped codepoints
new (ellipse + rotation dedup)	38,316 mapped codepoints

Rotation + reflection encoding (hand symbols)

For each hand `S{base}{fill}{rot}`, the last hex digit picks one of 16 orientations: 8 rotations of rot 0 plus their mirrors at the rot-8 offset. Diagonals (odd indices) form a parallel sub-family centred on rot 1. Non-hand symbols are not covered by this table — they keep their own outlines.

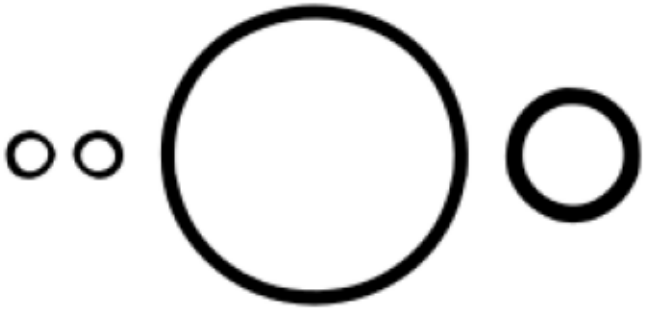
last hex	semantic orientation	composite source	transform applied
0	rot 0°	base (outline kept)	—
1	rot 45°	diag base (outline ke	—
2	rot 90°	← rot 0	rotate 90°
3	rot 135°	← rot 1	rotate 90°
4	rot 180°	← rot 0	rotate 180°
5	rot 225°	← rot 1	rotate 180°
6	rot 270°	← rot 0	rotate 270°
7	rot 315°	← rot 1	rotate 270°
8	mirror + rot 0°	← rot 0	mirror
9	mirror + rot 45°	← rot 1	mirror
A	mirror + rot 90°	← rot 0	mirror + rot 270°
B	mirror + rot 135°	← rot 1	mirror + rot 270°
C	mirror + rot 180°	← rot 0	mirror + rot 180°
D	mirror + rot 225°	← rot 1	mirror + rot 180°
E	mirror + rot 270°	← rot 0	mirror + rot 90°
F	mirror + rot 315°	← rot 1	mirror + rot 90°

Mirror reverses CCW→CW, which is why rot A's matrix is `mirror + rot 270°` even though the semantic is `mirror + rot 90°`.

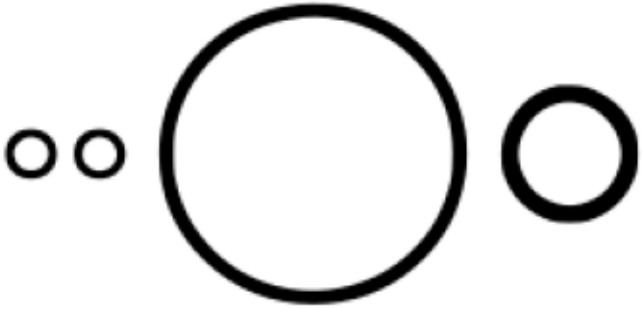
Ellipse optimization — circular glyphs

hand-traced cubic circles replaced by 4-segment Bezier ellipses

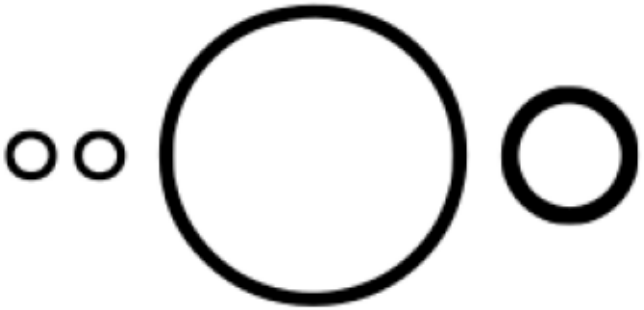
original (upstream OneD)



new (no optimisations)



new (ellipse + rotation dedup)



Rotation dedup — all 16 orientations of S1000{0..f}

even indices = composites of rot 0; odd = composites of rot 1

original (upstream OneD)

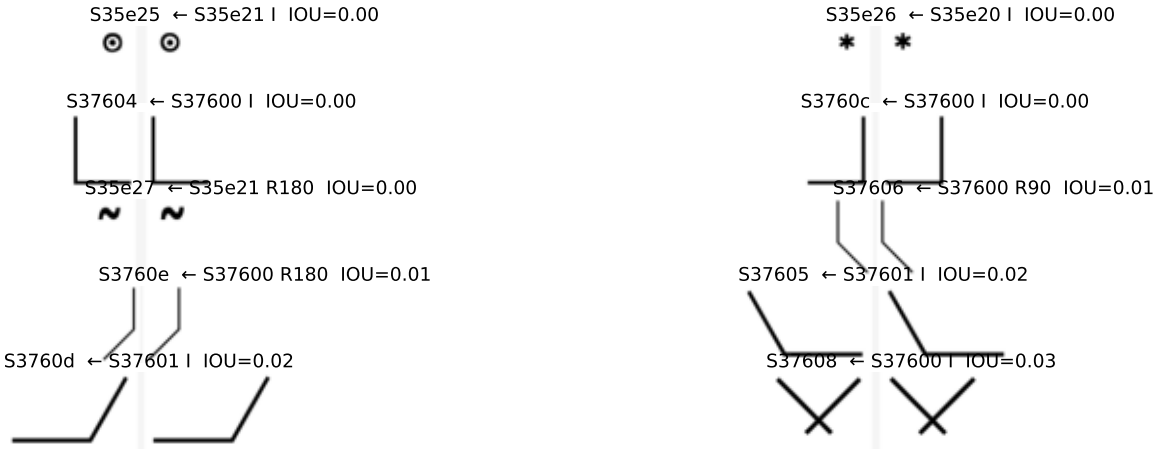
new (no optimisations)

new (ellipse + rotation dedup)

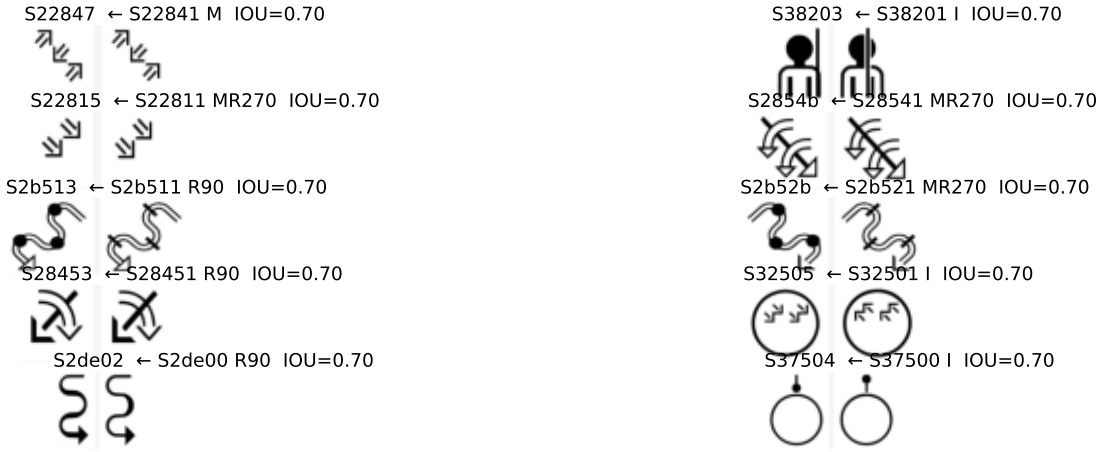
Threshold tuning — top vs bottom of the 0.60 cut

Top section: 10 most-certain rejects (lowest IOU among rejected — these stay as outlines).
Bottom section: 10 weakest accepts (lowest IOU among duplicates — these became composites).
If accepts at the bottom look wrong, the threshold is too low. If rejects at the top look correct, threshold could be lowered.

Most-certain rejects (lowest IOU among entries < 0.70):



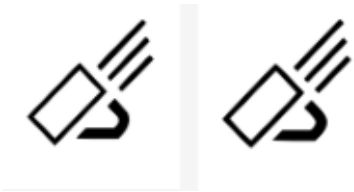
Weakest accepts (lowest IOU among entries ≥ 0.70):



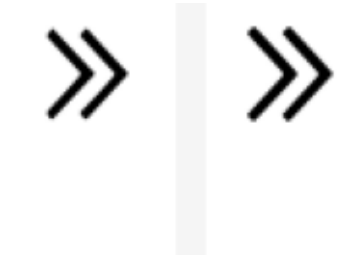
Known issues — one example per 0.05 IOU bucket

2 of ~38k glyphs render at IOU < 0.50 vs upstream; 12740 at IOU < 0.85. Each cell shows the upstream render on the left and our regenerated render on the right. Reading low→high IOU lets you eyeball a threshold at which the dedup composite is 'good enough'.

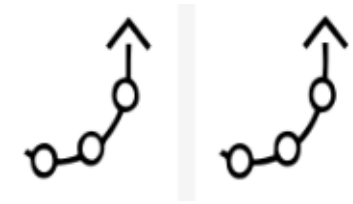
S1da39 IOU=0.44



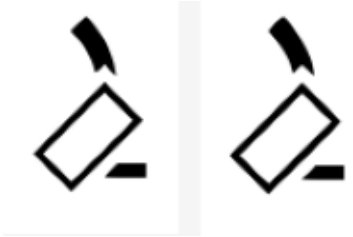
S22606 IOU=0.46



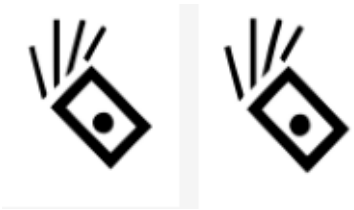
S21f17 IOU=0.54



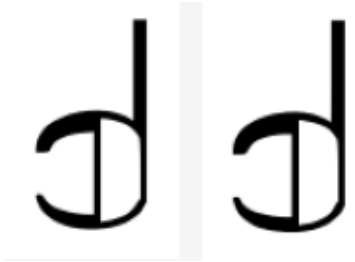
S16e37 IOU=0.59



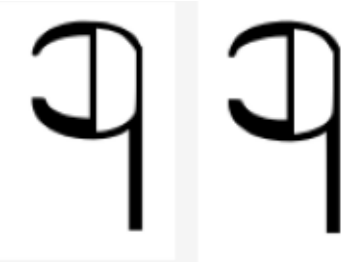
S1573f IOU=0.65



S10200 IOU=0.67



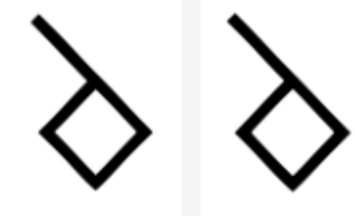
S1020c IOU=0.71



S10007 IOU=0.77

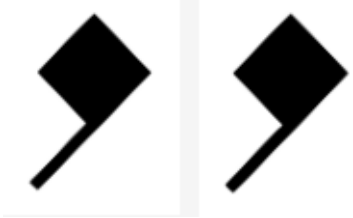
S10003 IOU=0.80

S10001 IOU=0.89



S1000b IOU=0.90

S10023 IOU=0.96



Threshold zoom — every example in IOU [0.40, 0.50)

2 sampled glyphs from the [0.40, 0.50) IOU band — the region where the eye-call between 'acceptable dedup' and 'reject' lives.
(Counts at fixed cuts: 1 glyphs < 0.44, 2 < 0.50.)

S1da39 IOU=0.44

S22606 IOU=0.46